

# Relatório de Execução - Heat Transfer (OpenMP

## GPU)

Este relatório detalha os resultados da execução do programa de simulação de transferência de calor (Heat Transfer) utilizando OpenMP. O objetivo foi explorar as diretivas de paralelização e movimentação de dados para a GPU (exercício do Slide 64).

### 1. Ambiente e Submissão

O job foi submetido ao cluster NPAD, utilizando a partição `gpu-8-v100`, utilizando a placa de vídeo NVIDIA V100. O código foi compilado utilizando o `nvc` com a flag `-mp=gpu` para habilitar o offloading de OpenMP para a GPU.

Foram avaliadas 3 versões do código para dois tamanhos de problema ( $N=1000$  e  $N=4000$ ), ambos com 100 timesteps:

1. **CPU (Baseline):** Paralelização multicore padrão utilizando `#pragma omp parallel for`.
2. **GPU Basic:** Paralelização com offloading para a GPU onde o mapeamento de dados ocorre internamente dentro do loop temporal, acarretando transferência de dados a cada passo de tempo (timestep).
3. **GPU Opt:** Paralelização com offloading otimizado, onde os dados são movidos para a GPU antes do loop temporal com `#pragma omp target enter data`, e recuperados apenas após o término do loop com `#pragma omp target exit data`.

### 2. Comparação de Desempenho

#### Resultados para $N = 1000$ (Grid $1000 \times 1000$ )

| Versão                | Tempo de Solve (s) | Tempo Total (s) | Erros (L2 Norm) |
|-----------------------|--------------------|-----------------|-----------------|
| <b>CPU (Baseline)</b> | 0.598s             | 0.626s          | 4.055e-10       |
| <b>GPU Basic</b>      | 0.830s             | 0.859s          | 4.055e-10       |
| <b>GPU Opt</b>        | 0.008s             | 0.351s          | 4.055e-10       |

### Resultados para $N = 4000$ (Grid 4000x4000)

| Versão                | Tempo de Solve (s) | Tempo Total (s) | Erros (L2 Norm) |
|-----------------------|--------------------|-----------------|-----------------|
| <b>CPU (Baseline)</b> | 17.510s            | 18.000s         | 9.458e-11       |
| <b>GPU Basic</b>      | 7.284s             | 7.809s          | 9.458e-11       |
| <b>GPU Opt</b>        | 0.131s             | 0.994s          | 9.458e-11       |

### 3. Análise e Movimentação de Dados

Ao analisar os resultados e os dados de profiling gerados com o `nsys`, destacam-se os seguintes pontos:

- **O Gargalo da Movimentação Constante:** Na versão **GPU Basic**, o tempo de “Solve” é dominado pela transferência de memória. Para  $N = 1000$ , a CPU Baseline chega a ser mais rápida que a GPU (0.598s vs 0.830s) devido ao alto *overhead* de copiar os arrays `u` e `u_tmp` via barramento PCIe repetidas vezes (100 vezes).
- **Otimização de Dados (GPU Opt):** Quando as diretivas `#pragma omp target enter/exit data` são utilizadas fora do laço temporal, as transferências ocorrem estritamente quando necessário. No profiling com `nsys` para  $N = 4000$ , observa-se:
  - CUDA `memcpy Host-to-Device`: 2 chamadas (128 MB cada, referente a `u` e `u_tmp`), gastando ~31ms.
  - CUDA `memcpy Device-to-Host`: 1 chamada (128 MB, referente a `u` final), gastando ~19ms.
  - A execução dos kernels matemáticos demorou em torno de ~129ms acumulado ao longo de 100 execuções (~1.29ms por iteração para uma matriz 4000x4000).
- **Ganho Absoluto de Desempenho:** Com a otimização de dados correta, a versão **GPU Opt** para a malha maior de 4000x4000 reduziu o tempo de computação de 17.5 segundos na CPU p**0.13 segundos** na GPU - um ganho impressionante (speedup de mais de 130x no Solve).

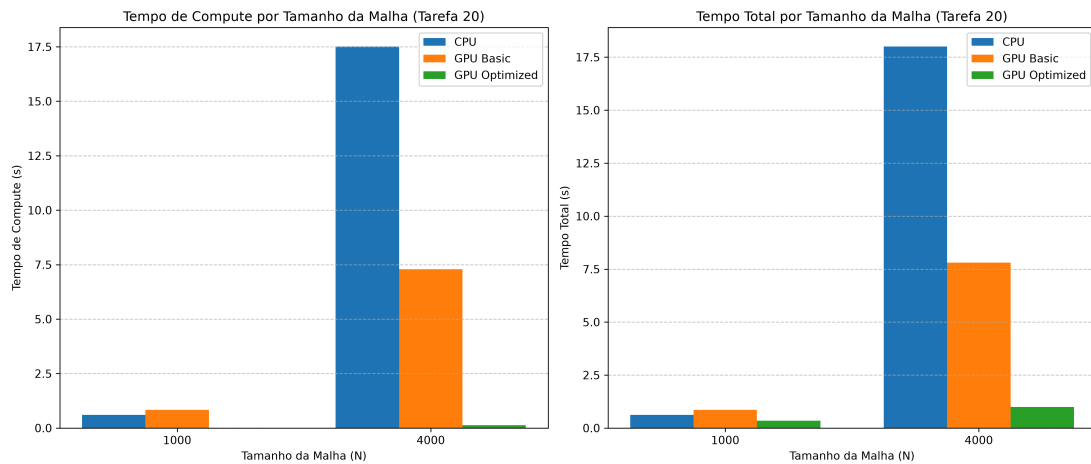


Figure 1: Gráfico de Barras - Tarefa 20

#### 4. Apêndice: Código da Solução Otimizada

```
// Em main():
// ...
// Copy data to the device
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])

// Run through timesteps under the explicit scheme
double tic = omp_get_wtime();
for (int t = 0; t < nsteps; ++t) {
    solve(n, alpha, dx, dt, u, u_tmp);
    // Pointer swap
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
double toc = omp_get_wtime();

// Copy data from the device
#pragma omp target exit data map(from: u[0:n*n])
// ...

// Em solve():
// ...
// Loop over the nxn grid
#pragma omp target
#pragma omp teams distribute parallel for collapse(2)
for (int i = 0; i < n; ++i) {
```

```

    for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] = r2 * u[i+j*n] +
            r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
            r * ((i > 0) ? u[i-1+j*n] : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
    }
}
// ...

```

### Script de Submissão (job.sh)

```

#!/bin/bash
#SBATCH --job-name=heat_omp_gpu
#SBATCH --time=0-0:15
#SBATCH --partition=gpu-8-v100
#SBATCH --gres=gpu:1
#SBATCH --mem=16G
#SBATCH --ntasks=1
#SBATCH --nodes=1
#SBATCH --cpus-per-task=4

# Load NVIDIA HPC SDK for nvc compiler and OpenMP offload support
export PATH=/opt/npad/shared/compilers/nvidia_hpc_sdk/24.11/
Linux_x86_64/24.11/compilers/bin:$PATH
export LD_LIBRARY_PATH=/opt/npad/shared/compilers/nvidia_hpc_sdk/24.11/
Linux_x86_64/24.11/compilers/lib:$LD_LIBRARY_PATH
# Add nsys to path
export PATH=/opt/npad/shared/compilers/nvidia_hpc_sdk/24.11/
Linux_x86_64/24.11/profilers/Nsight_Systems/bin:$PATH

make clean
make

echo "=== Executando heat_cpu (N=1000, 100 steps) ==="
./heat_cpu 1000 100

echo "=== Executando heat_gpu_basic (N=1000, 100 steps) ==="
./heat_gpu_basic 1000 100

echo "=== Executando heat_gpu_opt (N=1000, 100 steps) ==="

```

```

./heat_gpu_opt 1000 100

echo "=== Profiling heat_gpu_opt com nsys (N=1000, 100 steps) ==="
nsys profile --stats=true --force-overwrite=true -o heat_1000 ./
heat_gpu_opt 1000 100

echo ""
echo "=== Executando heat_cpu (N=4000, 100 steps) ==="
./heat_cpu 4000 100

echo "=== Executando heat_gpu_basic (N=4000, 100 steps) ==="
./heat_gpu_basic 4000 100

echo "=== Executando heat_gpu_opt (N=4000, 100 steps) ==="
./heat_gpu_opt 4000 100

echo "=== Profiling heat_gpu_opt com nsys (N=4000, 100 steps) ==="
nsys profile --stats=true --force-overwrite=true -o heat_4000 ./
heat_gpu_opt 4000 100

```